

Event Propagation

Capture → Target → Bubble | DOM & React reference

1. What is event propagation?

Event propagation is the journey an event takes through the DOM tree from the moment it fires to the moment it ends. It is not a single instant — it is a sequence of phases that visit every ancestor of the target element.

When you click a button, the click event does not fire only on the button. It travels through the entire DOM hierarchy above it, in a specific order, giving every ancestor a chance to react.

■ The big idea

Every event has a fixed journey through the DOM. You don't choose where it goes — you only choose **at which point during that journey** your handler is called.

2. The 3 phases

Every DOM event passes through three discrete phases, in order:

Phase	Direction	Description
1. Capturing	window → target	Event flows DOWN from the root to the target. Capture-phase listeners fire here.
2. Target	at the target	Event reaches the actual element it was fired on (e.target). Both kinds of listeners on the target fire.
3. Bubbling	target → window	Event flows UP from the target back to the root. Bubble-phase listeners fire here. Default phase.

Three discrete phases. Each ancestor is touched **twice** — once on the way down (capture), once on the way up (bubble).

```
window
■
■ PHASE 1: CAPTURING (going down)
▼
document
▼
html
▼
body
▼
div
▼
■■■■■■■■■■
■ BUTTON ■ ← PHASE 2: TARGET
■■■■■■■■■■
■
▲ PHASE 3: BUBBLING (going up)
■
div
▲
body
▲
...
▲
window
```

3. The key syntax — third argument of addEventListener

The whole secret of choosing the phase is in the third argument:

```
element.addEventListener(eventType, handler, useCapture);
// ^^^^^^^^^^^
// true → listen during CAPTURE phase
// false → listen during BUBBLE phase (default)

// Modern object form:
element.addEventListener('click', fn, { capture: true });
```

■ ■ Crucial mental model

true / false is NOT direction between elements. It just tells the browser **WHEN** during the event's journey to call your handler — on the way down (capture) or on the way up (bubble). The journey itself is fixed by the DOM.

4. Full example — see all 3 phases

HTML structure:

```
<div id="grandparent">
  <div id="parent">
    <button id="child">Click me</button>
  </div>
</div>
```

JavaScript — 5 listeners total:

```
const gp = document.getElementById('grandparent');
const p = document.getElementById('parent');
const c = document.getElementById('child');

// 2 listeners on grandparent
gp.addEventListener('click', () => console.log('1. grandparent CAPTURE'), true);
gp.addEventListener('click', () => console.log('5. grandparent BUBBLE'));

// 2 listeners on parent
p.addEventListener('click', () => console.log('2. parent CAPTURE'), true);
p.addEventListener('click', () => console.log('4. parent BUBBLE'));

// 1 listener on child (the target itself)
c.addEventListener('click', () => console.log('3. child (TARGET)'));
```

Click the child button. The console output is:

1. grandparent CAPTURE ← event going DOWN, capture phase
2. parent CAPTURE ← event going DOWN, capture phase
3. child (TARGET) ← event reaches the target
4. parent BUBBLE ← event going UP, bubble phase
5. grandparent BUBBLE ← event going UP, bubble phase

■ How to read this

The numbers in the strings (1–5) are just labels showing the firing order. Capture listeners on ancestors fire FIRST (top-down) because the event reaches ancestors on the way down before it gets to the target. Bubble listeners on ancestors fire LAST (bottom-up) because the event reaches them again on the way back up.

5. The common confusion — clearing it up

Many devs think *true* means "capturing FROM A TO B" — like a direction between two elements. That is wrong.

■ Correct mental model

true means: *"this listener listens during the capture phase."*

false (or omitted) means: *"this listener listens during the bubble phase."*

The phase is a property of the event's universal journey — not a relationship between two elements.

Analogy — the runner and buzzers

Imagine the click event is a runner going down a hallway, then back up. Every door (element) has a **front buzzer** (capture) and a **back buzzer** (bubble).

- Going DOWN: the runner hits each FRONT buzzer in order.
- Hits the target room.
- Going UP: the runner hits each BACK buzzer in order.

When you do `addEventListener('click', fn, true)` on element X, you're wiring `fn` to X's **front buzzer**. With `false` (or omitted), you're wiring `fn` to X's **back buzzer**. The runner always does the full route. Your handler only fires if you've wired it to a buzzer the runner passes.

6. Stopping propagation

Method	What it stops
<code>e.stopPropagation()</code>	Stops further travel through phases. Other listeners on the same element still run.
<code>e.stopImmediatePropagation()</code>	Stops travel + skips other listeners on the same element.
<code>e.preventDefault()</code>	Does NOT stop propagation. Only cancels the browser's default action (form submit, link nav, etc).

Visual difference:

```
parent.addEventListener('click', () => console.log('parent'));

child.addEventListener('click', (e) => {
  e.stopPropagation();
  console.log('child A');
});
child.addEventListener('click', () => console.log('child B'));

// Click child:
// child A
// child B ← still fires (same element)
// (parent never fires – propagation stopped)
```

```
child.addEventListener('click', (e) => {
  e.stopImmediatePropagation();
  console.log('child A');
});
child.addEventListener('click', () => console.log('child B'));

// Click child:
// child A
// (child B AND parent never fire – both blocked)
```

■ stopPropagation vs preventDefault

They are **independent**. You can use one, both, or neither.

stopPropagation = stops the event from traveling further.

preventDefault = cancels the browser's built-in behavior (form submit reload, link navigation).

7. Events that don't bubble

Most events bubble, but a few don't. Useful to know — they're a common bug source.

Doesn't bubble	Bubbling alternative
focus	focusin
blur	focusout

Doesn't bubble	Bubbling alternative
mouseenter	mouseover
mouseleave	mouseout
load, unload	—
scroll (most elements)	bubbles on document

■ React's secret

React **synthesizes bubbling** for onFocus and onBlur even though native focus/blur don't bubble. So in React you don't need focusin/focusout — just use onFocus/onBlur on a parent and it works.

8. Propagation in React

React mirrors the DOM model with its **SyntheticEvent** wrapper:

- Events bubble through the JSX tree, just like the DOM.
- Default phase is bubble (onClick).
- Capture phase = append **Capture** (onClickCapture).
- `e.stopPropagation()` halts the synthetic propagation.
- `e.nativeEvent.stopImmediatePropagation()` stops the native event too.

```
<div onClickCapture={() => console.log('parent capture')}
  onClick={() => console.log('parent bubble')}>
  <button onClickCapture={() => console.log('child capture')}
    onClick={() => console.log('child bubble')}>
    Click
  </button>
</div>

// Output when clicking the button:
// parent capture
// child capture
// child bubble
// parent bubble
```

React 17+ delegation gotcha

Since React 17, all events attach to the **root container** (e.g. `<div id="root">`), not document. This means `e.stopPropagation()` only halts React's synthetic tree walk — the underlying native event still bubbles up to document.

```
useEffect(() => {
  document.addEventListener('click', () => console.log('NATIVE'));
}, []);

return (
  <button onClick={(e) => {
    e.stopPropagation(); // stops React bubbling, but NOT native
    console.log('REACT');
  }}>Click</button>
);

// Output when clicking the button:
// REACT
// NATIVE ← still fires!
```

To stop the native event too:

```
onClick={(e) => {
  e.nativeEvent.stopImmediatePropagation();
}}
```

9. Real-world patterns built on propagation

Modal backdrop close

```
<div className="backdrop" onClick={onClose}>
<div className="modal" onClick={(e) => e.stopPropagation()}>
  /* clicks here don't bubble to backdrop -> modal stays open */
</div>
</div>
```

Card click + inner button (don't trigger card)

```
<div onClick={() => navigate(`/post/${id}`)}>
  <h3>{title}</h3>
  <button onClick={(e) => {
    e.stopPropagation();
    deletePost(id);
  }}>Delete</button>
</div>
```

Click-outside detection (relies on bubbling to document)

```
useEffect(() => {
  function handler(e) {
    if (!ref.current.contains(e.target)) setOpen(false);
  }
  document.addEventListener('mousedown', handler);
  return () => document.removeEventListener('mousedown', handler);
}, []);
```

Event delegation (one listener for many children)

```
<ul onClick={(e) => {
  const li = e.target.closest('li');
  if (li) selectItem(li.dataset.id);
}}>
  {items.map(i => <li key={i.id} data-id={i.id}>{i.name}</li>)}
</ul>
```

10. Interview Q&A

■ Walk me through event propagation in the DOM.

An event has 3 phases. **Capturing** — travels from window down to target. **Target** — at the element itself. **Bubbling** — travels from target back up to window. By default `addEventListener` listens on the bubble phase; pass `{ capture: true }` for the capture phase. Use `e.stopPropagation()` to halt further travel.

■ What's the difference between `stopPropagation` and `preventDefault`?

`stopPropagation` stops the event from continuing through phases — no more ancestor handlers fire. **`preventDefault`** cancels the browser's default action (form submit, link navigation) but doesn't affect propagation. They're independent — you can use one, both, or neither.

■ Difference between `stopPropagation` and `stopImmediatePropagation`?

Both stop further travel through phases. The difference: **`stopImmediatePropagation` also prevents other listeners on the same element** from running. Regular `stopPropagation` lets sibling listeners on the same element finish.

■ Why does `e.stopPropagation` in React not stop a native document listener?

Since React 17, React's synthetic event listener is at the root container, not document. By the time React calls your handler, the native event has already bubbled up through the real DOM. `stopPropagation` halts React's synthetic walk, but the native event continues to document. Use `e.nativeEvent.stopImmediatePropagation()`.

■ Which events don't bubble?

`focus`, `blur`, `mouseenter`, `mouseleave`, `load`, `unload`, and `scroll` on most elements. For `focus/blur` delegation, use `focusin/focusout`. React synthesizes bubbling for `onFocus/onBlur` even though native versions don't bubble.

■ Why is event delegation possible?

Because of bubbling. A click on a child travels up through every ancestor, so a single listener on a parent can handle clicks for all descendants. Saves memory, works for dynamically added children, scales well.

■ How would you detect a click outside a dropdown?

Add a `mousedown` listener on document (using `useEffect` in React). When fired, check if `e.target` is contained in the dropdown's ref element — if not, close. Works because of bubbling. Always remove the listener on cleanup.

■ When would you use the capture phase?

Rare in app code. Useful for: analytics/logging (catch all clicks before children can stop them), focus traps in modals, library code that needs first crack at events, situations where children might call `stopPropagation` and you must know about the event regardless.

11. TL;DR — one-page summary

■ The whole story

Event propagation = the 3-phase lifecycle of every DOM event:

1. **Capturing** — event flows from window down to the target.
2. **Target** — event reaches the element it was fired on.
3. **Bubbling** — event flows from target back up to window.

Default listeners run on bubble. Use `addEventListener(type, fn, { capture: true })` for capture. Stop further travel with `e.stopPropagation()` (or `stopImmediatePropagation` to also skip same-element siblings). `preventDefault` is independent — it cancels the browser's default action but doesn't affect propagation. Some events don't bubble (focus, blur, mouseenter, mouseleave). Bubbling is the foundation of event delegation, click-outside detection, and most real-world DOM interactivity.

One-line interview answer

■ If you only remember one sentence...

"Propagation is capture phase down → target → bubble phase up. Default listeners are on bubble. stopPropagation halts travel; preventDefault cancels the default action. They're separate concerns."